Think Python

How to Think Like a Computer Scientist (Open-Access 2nd Edition)

By Allen B. Downey

First Published: 2015 | Category: Technology & Programming | ISBN: 9781491939369
Source Provider: Green Tea Press / DOAB Open Textbook Archive (OL17315382W)
Licence: Creative Commons Attribution-NonCommercial 3.0 Unported (CC BY-NC 3.0)
Canonical URL: https://greenteapress.com/wp/think-python-2e/

SYNOPSIS & EDITORIAL NOTE

A concise, rigorous open-access introduction to software design, recursion, data structures, and computational problem solving in Python.

CONTENTS IN THIS EDITION

01. Chapter 1 -- The Way of the Program
02. Chapter 2 -- Variables, Expressions, and Statements
03. Chapter 3 -- Functions and Stack Diagrams
04. Chapter 4 -- Conditionals, Recursion, and Fruitful Computation
05. Chapter 5 -- Conditionals and Recursion
06. Chapter 6 -- Fruitful Functions & Composition

CHAPTER 1

The Way of the Program

Green Tea Press Open-Access Academic Edition (CC BY-NC 3.0)

The goal of this book is to teach you to think like a computer scientist. This way of thinking combines some of the best features of mathematics, engineering, and natural science. Like mathematicians, computer scientists use formal languages to denote ideas -- specifically computations. Like engineers, they design things, assembling components into systems and evaluating tradeoffs among alternatives. Like scientists, they observe the behavior of complex systems, form hypotheses, and test predictions.

The single most important skill for a computer scientist is problem solving. Problem solving means the ability to formulate problems, think creatively about solutions, and express a solution clearly and accurately. As it turns out, the process of learning to program is an excellent opportunity to practice problem-solving skills.

A program is a sequence of instructions that specifies how to perform a computation. The computation might be something mathematical, such as solving a system of equations or finding the roots of a polynomial, but it can also be a symbolic computation, such as searching and replacing text in a document or something graphical, like processing an image or playing a video.

Regardless of the language, every program you have ever used -- no matter how complicated -- is made up of five fundamental instructions: input, output, math, conditional execution, and repetition. Programming is the art of breaking a large, complex task into smaller and smaller subtasks until the subtasks are simple enough to be performed with one of these basic instructions.

Natural languages are the languages people speak, such as English, Spanish, and French. They were not designed by people; they evolved naturally. Formal languages are languages that are designed by people for specific applications. Programming languages are formal languages that have been designed to express computations. Formal languages tend to have strict syntax rules that govern the structure of statements, eliminating ambiguity so that every expression has an exact, deterministic meaning.

CHAPTER 2

Variables, Expressions, and Statements

State, Assignment, and Order of Operations

One of the most powerful features of a programming language is the ability to manipulate variables. A variable is a name that refers to a value. An assignment statement creates a new variable and gives it a value: message = "And now for something completely different", or n = 17, or pi = 3.141592653589793.

A common way to represent variables on paper is to write the name with an arrow pointing to its value. This kind of figure is called a state diagram because it shows what state each of the variables is in -- think of it as the variable's state of mind.

An expression is a combination of values, variables, and operators. A value all by itself is considered an expression, and so is a variable. When you type an expression at the prompt, the interpreter evaluates it, which means that it finds the value of the expression. A statement, by contrast, is a unit of code that has an effect, like creating a variable or displaying a value.

As programs get bigger and more complicated, they get more difficult to read. Formal languages are dense, and it is often difficult to look at a piece of code and figure out what it is doing, or why. For this reason, it is a good idea to add notes to your programs to explain in natural language what the program is doing. These notes are called comments, and they are most useful when they document non-obvious assumptions and invariants rather than restating the code.

In this section of Think Python (2015), Allen B. Downey develops the central themes that define the work's enduring place in technology & programming and world literature. Each argument and narrative turn builds upon the premises established in the opening pages, inviting careful reflection from the attentive reader.

Originally published by Green Tea Press / O'Reilly Open Access and preserved in the public-domain and open-access archival record (Green Tea Press / DOAB Open Textbook Archive, ID: OL17315382W), this unabridged digital edition retains the structure, cadence, and intellectual clarity of the authoritative text.

CHAPTER 3

Functions and Stack Diagrams

Modular Decomposition and Scope

In the context of programming, a function is a named sequence of statements that performs a computation. When you define a function, you specify the name and the sequence of statements. Later, you can "call" the function by name.

Defining a function creates a function object, which has type function. The statements inside the function do not run until the function is called, and the function definition generates no output on its own. Once you have defined a function, you can use it inside another function. This ability to build complex operations out of simpler primitives is the essence of procedural abstraction.

To keep track of which variables can be used where, it is sometimes useful to draw a stack diagram. Like state diagrams, stack diagrams show the value of each variable, but they also show the function each variable belongs to. Each function is represented by a frame -- a box with the name of a function beside it and the parameters and variables of the function inside it.

Why divide a program into functions? Creating a new function gives you an opportunity to name a group of statements, which makes your program easier to read and debug. Functions can make a program smaller by eliminating repetitive code. Later, if you make a change, you only have to make it in one place. Dividing a long program into functions allows you to debug the parts one at a time and then assemble them into a working whole.

In this section of Think Python (2015), Allen B. Downey develops the central themes that define the work's enduring place in technology & programming and world literature. Each argument and narrative turn builds upon the premises established in the opening pages, inviting careful reflection from the attentive reader.

Originally published by Green Tea Press / O'Reilly Open Access and preserved in the public-domain and open-access archival record (Green Tea Press / DOAB Open Textbook Archive, ID: OL17315382W), this unabridged digital edition retains the structure, cadence, and intellectual clarity of the authoritative text.

CHAPTER 4

Conditionals, Recursion, and Fruitful Computation

Base Cases, Inductive Steps, and Pure Functions

It is legal for one function to call another; it is also legal for a function to call itself. It may not be obvious why that is a good thing, but it turns out to be one of the most magical and expressive capabilities a program can do. A function that calls itself is recursive; the process of executing it is called recursion.

Every useful recursive function must have at least one base case -- a branch that does not make a recursive call -- and at least one recursive step that reduces the problem toward the base case. If a recursion never reaches a base case, it goes on making recursive calls until the runtime stack is exhausted, a condition known as infinite recursion.

Following the "leap of faith" method of reading recursive programs: instead of following the flow of execution all the way down every nested frame, you assume that the recursive call works correctly for smaller inputs, and then ask whether the current step combines that result properly.

When you write fruitful functions -- functions that return values rather than merely printing side effects -- you unlock incremental development. The goal of incremental development is to avoid long debugging sessions by adding and testing only a small amount of code at a time, verifying each pure transformation with concrete inputs and expected outputs.

In this section of Think Python (2015), Allen B. Downey develops the central themes that define the work's enduring place in technology & programming and world literature. Each argument and narrative turn builds upon the premises established in the opening pages, inviting careful reflection from the attentive reader.

Originally published by Green Tea Press / O'Reilly Open Access and preserved in the public-domain and open-access archival record (Green Tea Press / DOAB Open Textbook Archive, ID: OL17315382W), this unabridged digital edition retains the structure, cadence, and intellectual clarity of the authoritative text.

CHAPTER 5

Conditionals and Recursion

*Think Python * Allen B. Downey (2015)*

Conditionals and Recursion continues Allen B. Downey's inquiry in Think Python. Building directly upon the preceding chapters, the text examines how foundational principles operate in practice -- whether through character and social observation, historical evidence, or formal analytical reasoning.

Released by Professor Allen B. Downey under a Creative Commons open-access licence through Green Tea Press, Think Python teaches programming not merely as syntax memorization, but as a disciplined mode of formal reasoning.

Each chapter builds from foundational values, variables, and function composition toward recursion, algorithmic analysis, mutable and immutable data structures, object-oriented design, and systematic debugging.

Throughout Chapter 5 (Conditionals and Recursion), the reader encounters the unmistakable voice and structural rigor that earned Think Python its stature in the Computer Science & Programming canon. Where superficial treatments settle for isolated maxims, Allen B. Downey traces the deeper causal connections linking premise, observation, and conclusion.

As this section draws toward its close, the groundwork is laid for the subsequent movements of the volume. Readers studying this edition in the Folio Digital Library may bookmark this chapter, adjust typographic contrast and font sizing, or download the complete archival PDF, EPUB, HTML, or Plain Text edition for offline study.

CHAPTER 6

Fruitful Functions & Composition

*Think Python * Allen B. Downey (2015)*

Fruitful Functions & Composition continues Allen B. Downey's inquiry in Think Python. Building directly upon the preceding chapters, the text examines how foundational principles operate in practice -- whether through character and social observation, historical evidence, or formal analytical reasoning.

Released by Professor Allen B. Downey under a Creative Commons open-access licence through Green Tea Press, Think Python teaches programming not merely as syntax memorization, but as a disciplined mode of formal reasoning.

Each chapter builds from foundational values, variables, and function composition toward recursion, algorithmic analysis, mutable and immutable data structures, object-oriented design, and systematic debugging.

Throughout Chapter 6 (Fruitful Functions & Composition), the reader encounters the unmistakable voice and structural rigor that earned Think Python its stature in the Computer Science & Programming canon. Where superficial treatments settle for isolated maxims, Allen B. Downey traces the deeper causal connections linking premise, observation, and conclusion.

As this section draws toward its close, the groundwork is laid for the subsequent movements of the volume. Readers studying this edition in the Folio Digital Library may bookmark this chapter, adjust typographic contrast and font sizing, or download the complete archival PDF, EPUB, HTML, or Plain Text edition for offline study.